DenseNet-121 Training Configuration

Official DenseNet repository audit and practical configuration for a 6 GB RTX 4050 Laptop GPU • Chest X-ray multi-label classification • 512 × 512 inputs • training from scratch • 50-epoch budget

1. Official source and scope

Primary source: liuzhuang13/DenseNet (original DenseNet repository). Its README gives the ImageNet command `-batchSize 256 -nEpochs 90 -depth 121 -growthRate 32 -nGPU 4 -nThreads 16 -optMemory 3`. The original repository is Torch7/Lua, not a ready-to-run PyTorch training script. This document maps its published configuration to a PyTorch implementation while clearly flagging changes.

Important: the official values were tuned for ImageNet-scale training and multi-GPU hardware. They are not automatically optimal for TBX11K or for 512 × 512 images on a 6 GB GPU.

2. Parameter-by-parameter comparison

Parameter	Official repository / paper reference	Use in your experiment (copy/adapt)	Why / notes
Architecture	DenseNet-BC, depth 121	DenseNet-121 / DenseNet-BC-121 equivalent	Keep the 121-layer DenseNet-BC structure. Torchvision densenet121 is a compatible architecture implementation, but not the same Torch7 codebase.
Growth rate	k = 32	32	Architecture parameter; keep unchanged.
Bottleneck / compression	BC: 1×1 bottleneck layers; transition compression 0.5	Keep standard DenseNet-121 BC design	Do not accidentally instantiate a non-BC DenseNet.
Input size	224×224 ImageNet evaluation/training convention	512×512 (project requirement)	Adaptation, not official. Higher spatial memory/compute; 512^2 has about 5.2× as many pixels as 224^2.
Initialization	Original paper uses its prescribed random initialization; repository trains the ImageNet model from scratch	Random initialization; no ImageNet weights	Matches your explicit from-scratch requirement. Do not load torchvision DEFAULT weights.
Dataset / task	ImageNet, 1,000-way single-label classification	Your fixed TB chest-X-ray split; 2-output multilabel task	Labels are Active TB and Obsolete Pulmonary TB; combinations 00, 10, 01, 11 are possible.
Output layer	1,000 class logits	2 logits	One independent logit per TB label.
Loss	ImageNet single-label cross-entropy objective	BCEWithLogitsLoss	Required for multilabel targets. Encode targets as two floats, e.g. 11 → [1,1], not one four-class label.
Batch size	256	Start at 4; lower to 2 if OOM	256 is not realistic on 6 GB at 512^2. AMP and gradient accumulation can provide a larger effective batch.
Effective batch	256 per original command across 4 GPUs (global batch semantics depend on implementation)	Optional accumulation to effective 16 (e.g. batch 4 × 4 accumulation steps)	Suggested only; validate memory and optimizer-step accounting.
Epochs	90	50 fixed epochs (requested)	This is a deliberate deviation. Fifty epochs do not guarantee convergence; plot training/validation curves.
Optimizer	SGD	SGD	Copy the optimizer family rather than replacing it with Adam without an experiment-based reason.
Learning rate	0.1 initial for ImageNet recipe	Start with 0.01 as a conservative 1-GPU/small-batch adaptation	Official 0.1 was used with batch 256. Applying it unchanged to batch 2–4 may destabilize training.

Parameter	Official repository / paper reference	Use in your experiment (copy/adapt)	Why / notes
LR schedule	Multiply LR by 0.1 at epochs 30 and 60 (ImageNet recipe)	For 50 epochs: ×0.1 at epoch 30; optionally evaluate a second drop at 40 as a project-specific variant	The epoch-40 drop is suggested, not official. For strict schedule fidelity, only drop at epoch 30 within a 50-epoch run.
Momentum	0.9; Nesterov momentum in original paper recipe	0.9, Nesterov=True	Keep unchanged.
Weight decay	0.0001	0.0001	Copy. Check implementation conventions for weight decay on normalization parameters if claiming strict reproduction.
GPU count	4 GPUs	1 GPU: RTX 4050 Laptop, 6 GB VRAM	Adapted; no multi-GPU setup assumed.
Data-loader workers	16 threads	Start with 4 workers on Windows; adjust after testing	Practical Windows/RAM recommendation, not official.
Memory optimization	optMemory=3 (custom Torch7 implementation option)	No direct equivalent; use AMP + small batch + gradient accumulation + channels_last only if tested	Do not claim Torch7 optMemory=3 is enabled in PyTorch.
Precision	Not specified by the cited README command	CUDA autocast FP16 + GradScaler (or current torch.amp API)	Suggested memory reduction. Validate numerical stability.
Image normalization	ImageNet pipeline conventions for original ImageNet task	Compute/choose and document normalization for chest X-rays; use consistent train/val/test transforms	From-scratch training does not require ImageNet pretrained weights or ImageNet mean/std. Convert grayscale to 3 channels if using standard torchvision DenseNet input.
Augmentation	ImageNet recipe has its own training augmentation; exact transforms depend on data pipeline	Conservative, clinically valid train-only augmentation; none on validation/test	Do not invent aggressive flips/rotations without considering laterality and clinical validity.
Validation/checkpoint	ImageNet recipe reports benchmark metrics; exact project checkpoint policy is task-specific	Save best validation macro-F1; save last checkpoint too	Choose the best epoch using validation only, not test scores.
Test metrics	ImageNet top-1/top-5 error	Per-label precision/recall/F1, macro-F1, AUROC if supported, exact-match accuracy, confusion matrices per label	Use metrics appropriate for imbalanced multilabel TB classification.
Random seed / split	Not specified in the quoted README command	Use your already-established split and seed 42; log manifest paths	Do not create a new split for this model if comparing baselines.

3. Recommended configuration to implement

This is the recommended starting point for your project, not a verbatim reproduction of the original ImageNet run.

Setting	Recommended value	Status
Model	DenseNet-121 BC architecture; 2 output logits	Architecture adapted to task
Weights	Random initialization / train from scratch	Required by your project
Input	512 × 512; 3-channel tensor	Project requirement
Epoch budget	50	Project choice
Batch size	4; fallback 2 if CUDA OOM	Hardware adaptation
Gradient accumulation	4 steps if batch 4, giving nominal effective batch 16	Optional suggestion
Optimizer	SGD(lr=0.01, momentum=0.9, nesterov=True, weight_decay=1e-4)	Adapted LR; other values follow reference
Scheduler	Step LR ×0.1 at epoch 30; optionally test another drop at 40 separately	Epoch 30 follows reference; epoch 40 is optional
Loss	BCEWithLogitsLoss	Required for two-output multilabel targets
Mixed precision	FP16 autocast + GradScaler on CUDA	Suggested for VRAM
Data loading	num_workers=4 initially; pin_memory=True on CUDA	Windows-friendly starting point
Selection	Best validation macro-F1 checkpoint	Project protocol
Final test	Run once after selecting checkpoint; report per-label and macro metrics	Project protocol

4. Implementation checks before training

• **Targets:** Each sample must yield a two-element target [active_tb, obsolete_tb] in $\{0,1\}^2$. Use BCEWithLogitsLoss on raw logits; do not apply sigmoid before the loss.

• **Split:** Reuse the existing fixed train/validation/test manifests. Check that patient-level or duplicate-image leakage is not present if identifiers allow this.

• **Thresholds:** For per-label predictions, choose thresholds using validation data only. Report thresholds and include a standard 0.5-threshold result if useful for comparison.

• **Memory test:** Run a single forward/backward pass at 512 × 512 before committing to 50 epochs. If it OOMs, lower batch size first; do not silently reduce image resolution.

• **From scratch:** In Torchvision, instantiate densenet121(weights=None). Replace classifier with nn.Linear(in_features, 2). Confirm all backbone parameters are trainable.

• **Fair comparison:** Keep split, resolution, label mapping, preprocessing, augmentation policy and metric definitions consistent across DenseNet-121 and ResNet-101 baselines.

5. What can and cannot be called “official”

Copy directly where meaningful: DenseNet-BC-121 architecture, growth rate 32, SGD, momentum 0.9/Nesterov, weight decay 1e-4, and the original 30/60 LR-decay milestones. **Do not copy blindly:** batch size 256, four GPUs, 16 workers, 224×224 input, 90 epochs, ImageNet’s 1,000-class output, or its single-label loss. **Not portable:** optMemory=3 is a custom option in the original Torch7 code and has no direct switch in ordinary Torchvision DenseNet.

Convergence note: 50 epochs is a budget, not proof of convergence. If validation macro-F1 is still rising at epoch 50, say so in the report; do not claim the model converged solely because the planned epoch count finished.

6. Sources

1. Original DenseNet repository README and ImageNet command: https://github.com/liuzhuang13/DenseNet
2. Torchvision DenseNet implementation: https://github.com/pytorch/vision/blob/main/torchvision/models/densenet.py
3. Torchvision DenseNet-121 model documentation and pretrained inference transform: https://docs.pytorch.org/vision/main/models/generated/torchvision.models.densenet121.html
4. Huang et al., “Densely Connected Convolutional Networks,” CVPR 2017: https://arxiv.org/abs/1608.06993